

Recenzja kodu Projektu na przedmiot Programowanie Obiektowe

Projekt Aplikacja Bankowa wykonany przez Jakuba Kulińskiego i Oliwera Millera

Projekt Aplikacji Bankowej składa się z klasy głównej AplikacjaBankowa, która jest odpowiedzialna za komunikację z użytkownikiem poprzez interfejs konsolowy oraz klasy wykonawczej BankService, która jest odpowiedzialna za dodawanie kont i użytkowników (poprzez wywoływanie odpowiednich funkcji klas Repozytoriów odpowiedzialnych za komunikację z bazami danych) oraz za logowanie się użytkownika.

1. Wzorce projektowe wykorzystane w projekcie:

- **Singleton**

Wzorec Singleton został zastosowany w BankService. Ta klasa zawiera metodę statyczną getInstance(), która zapewnia, że może zostać utworzona tylko i wyłącznie jedna jej instancja, ponieważ wystarczy jedna klasa odpowiedzialna za sterowanie działaniami żądanymi przez użytkownika.

- **Factory**

Zastosowanie wzorca Factory Method jest widoczne w klasie RepoKonta. To właśnie ona pełni rolę fabryki poprzez tworzenie różnych rodzajów kont. W zależności od wartości zmiennej typ, decyduje czy zostanie utworzone konto oszczędnościowe, czy osobiste. Przewiduje również przypadek błędu, jakim może być nieznany typ konta określony przez klienta. Ogólnym produktem jest interfejs Konto, który definiuje wspólne zachowania dla wszystkich typów kont, takie jak: wpłata() i wypłata(). Rolę konkretnych produktów pełnią klasy dziedziczące po Konto, czyli KontoOszczednosciowe i KontoOsobiste.

2. Elementy programowania obiektowego wykorzystane w projekcie

- **Polimorfizm i dziedziczenie:** zastosowanie polimorfizmu widać w przypadku klas KontoOszczędnościowe oraz KontoOsobiste, bowiem każda z tych klas ma swoją implementację dziedziczonych metod z interfejsu Konto.
- **Enkapsulacja:** w projekcie widać zastosowanie publicznych geterów i seterów do prywatnych zmiennych, np w klasach: Konto, KontoOsobiste, KontoOszczednosciowe, Transakcja, RepoKonta, BankService.
- **Agregacja:** można ją zaobserwować z obiektami Typu Konto, które wywołując swoje metody tworzą obiekty klasy Transakcja do celów wykonania transakcji.

3. Połączenie z bazami danych

W projekcie wykorzystano bazy danych SQL, które tworzone są za pomocą dedykowanych klas - BazaKont, BazaKlientow, BazaHistoria, TESTBazaKont. Są one odpowiednio odpowiedzialne za konta, klientów, historię transakcji oraz testowanie aplikacji. Poszczególne bazy danych są połączone przez dedykowaną każdej bazie klasę Connector odpowiedzialną za połączenie z bazą danych SQL oraz klasy Repo (skrót od Repository), które zapewniają metody do pracy z bazą danych. Poszczególne operacje, które mogą wykonać klienci z poziomu kont, jak przelewy, wypłaty, wpłaty, stworzenie nowego konta, wygenerowanie wyciągu dla konta odbywają się za pomocą wywoływania odpowiednich funkcji z odpowiedniej klasy Repo. Jest to też spełnieniem pierwszej zasady SOLID - Single Responsibility. Jest osobna funkcja tworząca bazę danych, osobna do tworzenia połączenia, osobna do wykonywania operacji na bazie danych.

4. Unit tests

Aplikacja posiada dodatkowo klasę do przeprowadzania testów działania systemu, które polegają na wykonywaniu operacji na sztucznych kontach utworzonych w sztucznej bazie danych.

5. [Diagram UML](#) jest aktualny i odzwierciedla strukturę aplikacji, chociaż można się w nim pogubić ze względu na jego rozmiary.
6. Link do githuba <https://github.com/kulaBula/PO/tree/main/PO%20z%20unity%20tests>